

AIAA 3111 - 数据挖掘导论

第3讲：模式挖掘：基本概念与方法

Dr. Zhidan Liu



香港科技大学(广州)
THE HONG KONG
UNIVERSITY OF SCIENCE AND
TECHNOLOGY (GUANGZHOU)

第3-4讲：模式挖掘两讲

第1部分（80分钟）	第2部分（80分钟）	第4讲
事务、支持度、规则 Apriori 与向下封闭性	ECLAT 与 FP-Growth 闭模式与极大模式 模式评估	多层与多维模式 基于约束、序列与子图挖掘

本周建立共同基础。
第4周扩展模式类型。

学习成果

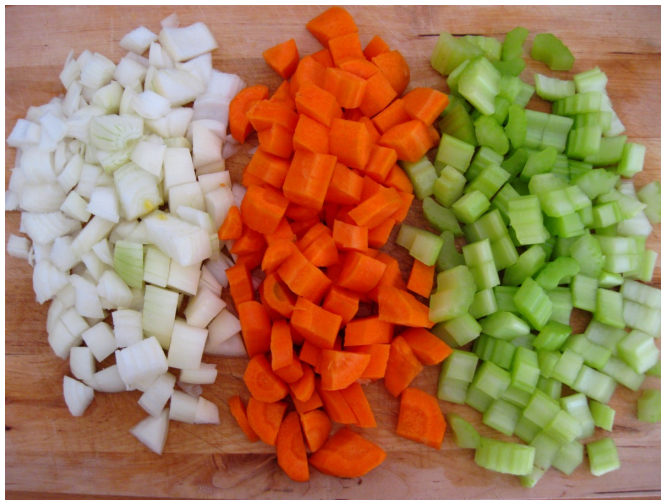
表示	计算	推演	评估
将记录转换为事务、项、项集和规则	计算支持度、置信度、提升度与 χ^2	在小数据上推演 Apriori、ECLAT 与 FP-Growth	选择挖掘方法，并判断规则是否有用

应能计算、推演、比较并解释。

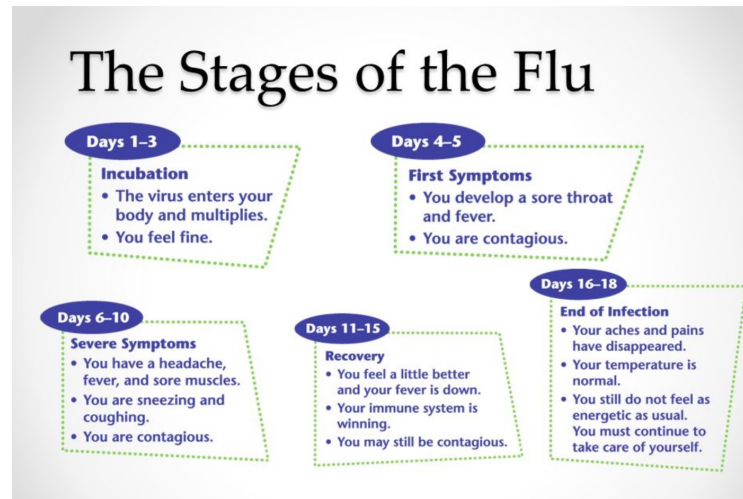
什么是模式？

□ 什么是模式？

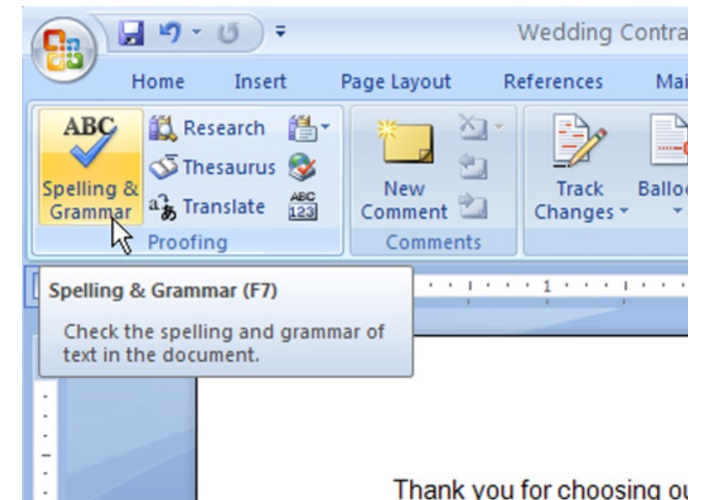
- 模式：在数据集中频繁共同出现（或强相关）的一组项、子序列或子结构
- 模式刻画数据集内在而重要的性质



频繁项集



频繁序列



频繁结构

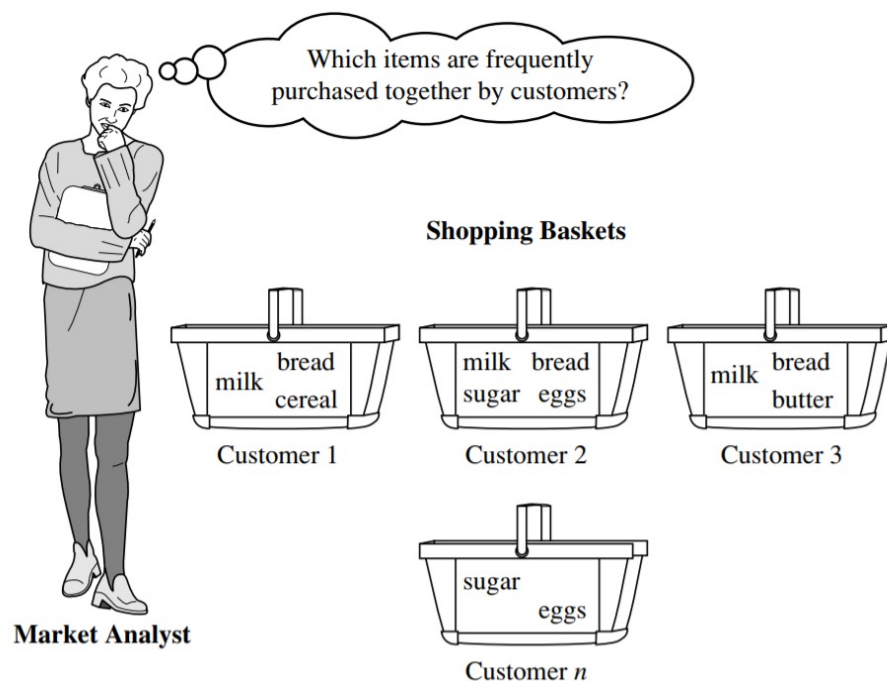
模式类型与课程范围

模式形态	示例	覆盖范围
频繁项集	订阅用户, 早晨, 中心站	第3周: 完整讲授
关联规则	早晨 $\Rightarrow$ 订阅用户	第3周: 完整讲授
序列模式	搜索 $\Rightarrow$ 比较 $\Rightarrow$ 购买	第4周
结构模式	反复出现的子图	第4周
多层或带约束的模式	交通 $\Rightarrow$ 自行车 $\Rightarrow$ 电动自行车	第4周

项集为更高级的模式类型提供基本语言。

什么是模式发现？

- 模式发现：从海量数据中揭示模式
- 它可以回答如下问题：
 - 哪些商品经常被一起购买？
 - 购买 iPad 之后通常还会买什么？



模式发现：为何重要？

- 许多核心数据挖掘任务的基础

- 关联、相关与因果分析
- 挖掘序列、结构（如子图）模式
- **分类：基于判别性模式的分析**
- **聚类分析：基于模式的子空间聚类**

- 多种数据类型：时空、多媒体、时间序列与流数据

- 广泛应用

- 购物篮分析、交叉营销、目录设计、促销分析、Web 日志分析、生物序列分析

基本概念：事务数据库

- 事务数据库 (TDB)
 - 每条事务都有一个标识符，称为 TID。
 - 售出各项也可以附带计数

TID	购买的项
1	啤酒, 坚果, 尿布
2	啤酒, 咖啡, 尿布
3	啤酒, 尿布, 鸡蛋
4	坚果, 鸡蛋, 牛奶
5	坚果, 咖啡, 尿布, 鸡蛋, 牛奶

基本概念：k-项集及其支持度

- 项集：由一个或多个项组成的集合

$$I = \{I_1, I_2, \dots, I_m\}$$

- k-项集：含有 k 个项的项集：

$$X = \{x_1, \dots, x_k\}$$

例. {Beer, Nuts, Diaper} 是 3-项集

- 绝对支持度 (计数)**

$\text{sup}\{X\}$ = 项集 X 出现的次数

例. $\text{sup}\{\text{Beer}\} = 3$

例. $\text{sup}\{\text{Diaper}\} = 4$

例. $\text{sup}\{\text{Beer}, \text{Diaper}\} = 3$

例. $\text{sup}\{\text{Beer}, \text{Eggs}\} = 1$

TID	购买的项
1	啤酒, 坚果, 尿布
2	啤酒, 咖啡, 尿布
3	啤酒, 尿布, 鸡蛋
4	坚果, 鸡蛋, 牛奶
5	坚果, 咖啡, 尿布, 鸡蛋, 牛奶

相对支持度

$s\{X\}$ = 包含 X 的事务所占比例 (即事务包含 X 的概率)

例. $s\{\text{Beer}\} = 3/5 = 60\%$

例. $s\{\text{Diaper}\} = 4/5 = 80\%$

例. $s\{\text{Beer}, \text{Eggs}\} = 1/5 = 20\%$

选择事务单位

事务单位	项可以表示	能回答的问题
一次出行	会员类型、时段、站点、时长类别	哪些出行情境会共现？
一个用户日	到访站点、使用类别	同一天会发生哪些活动？
一个站点小时	天气、需求水平、活动状态	哪些运营条件会共现？

注：出行次数等数量可保持为数值，也可转换为如 HighDemand 这样的项。

改变事务单位会改变支持度的含义。

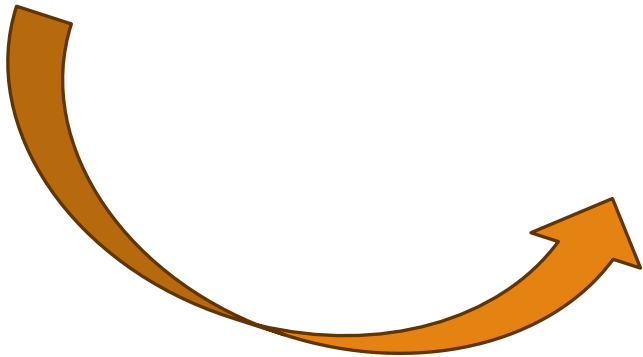
共享单车记录作为挖掘示例

属性				
trip_id	会员	时段	起始站点	时长
T1	订阅用户	早晨	中心站	短时
T2	订阅用户	早晨	中心站	短时
T3	订阅用户	早晨	河滨站	短时
T4	临时用户	周末	河滨站	长时
T5	临时用户	周末	博物馆站	长时
T6	临时用户	周末	河滨站	长时
T7	订阅用户	傍晚	中心站	短时
T8	临时用户	周末	博物馆站	长时

表中存储的是属性。模式挖掘把选定标签当作项。

共享单车事务数据库

trip_id	member	period	start station	duration
T1	Subscriber	Morning	Central	Short
T2	Subscriber	Morning	Central	Short
T3	Subscriber	Morning	Riverside	Short
T4	Casual	Weekend	Riverside	Long
T5	Casual	Weekend	Museum	Long
T6	Casual	Weekend	Riverside	Long
T7	Subscriber	Evening	Central	Short
T8	Casual	Weekend	Museum	Long



TID	出现的项
T1	{订阅用户, 早晨, 中心站, 短时}
T2	{订阅用户, 早晨, 中心站, 短时}
T3	{订阅用户, 早晨, 河滨站, 短时}
T4	{临时用户, 周末, 河滨站, 长时}
T5	{临时用户, 周末, 博物馆站, 长时}
T6	{临时用户, 周末, 河滨站, 长时}
T7	{订阅用户, 傍晚, 中心站, 短时}
T8	{临时用户, 周末, 博物馆站, 长时}

事务包含关系与支持度

□ 若事务包含 x 中的全部项，则该事务计入 x 的支持度。允许有额外项。

T1 = {订阅用户, 早晨, 中心站, 短时}

T1 所支持的例子

- {订阅用户}
- {早晨, 中心站}
- {订阅用户, 早晨, 短时}

T1 不支持的例子

- {临时用户}
- {中心站, 长时}
- {订阅用户, 河滨站}

含 k 个项的事务支持 $2^k - 1$ 个非空子集。

基本概念：频繁项集（模式）

- 若项集（或模式） x 的支持度不低于 minsup 阈值 σ ，则称 x 为频繁的
- 设 $\sigma = 50\%$ （ σ : minsup 阈值），对给定的 5 条事务数据集
 - 全部频繁 1-项集：
 - Beer: 3/5 (60%); Nuts: 3/5 (60%); Diaper: 4/5 (80%); Eggs: 3/5 (60%)
 - 全部频繁 2-项集：
 - {Beer, Diaper}: 3/5 (60%)
 - 全部频繁 3-项集？
 - 无



TID	购买的项
1	啤酒, 坚果, 尿布
2	啤酒, 咖啡, 尿布
3	啤酒, 尿布, 鸡蛋
4	坚果, 鸡蛋, 牛奶
5	坚果, 咖啡, 尿布, 鸡蛋, 牛奶

为什么左图这些项集构成任意 k 的完整频繁 k -项集（模式）集合？（请核对）

- 观察：需要一种高效搜索方法，在剪枝不可能候选的同时保持完备性。

最小支持度阈值

TID	Items present
T1	{Subscriber, Morning, Central, Short}
T2	{Subscriber, Morning, Central, Short}
T3	{Subscriber, Morning, Riverside, Short}
T4	{Casual, Weekend, Riverside, Long}
T5	{Casual, Weekend, Museum, Long}
T6	{Casual, Weekend, Riverside, Long}
T7	{Subscriber, Evening, Central, Short}
T8	{Casual, Weekend, Museum, Long}

降低 minsup 会提高发现灵敏度，同时增加计算代价

。

minsup 计数	保留的模式	主要风险
4	非常常见的二元组与三元组	漏掉有用但不那么常见的模式
3	单车示例中的核心模式	平衡的教学设定
2	博物馆相关模式也会出现	候选更多、冗余更多
1	每一个观察到的组合	模式爆炸

快速检查：支持度

TID	Items present
T1	{Subscriber, Morning, Central, Short}
T2	{Subscriber, Morning, Central, Short}
T3	{Subscriber, Morning, Riverside, Short}
T4	{Casual, Weekend, Riverside, Long}
T5	{Casual, Weekend, Museum, Long}
T6	{Casual, Weekend, Riverside, Long}
T7	{Subscriber, Evening, Central, Short}
T8	{Casual, Weekend, Museum, Long}

问题：对 $X = \{\text{Casual, Weekend, Long}\}$, $s(X)$ 是多少？

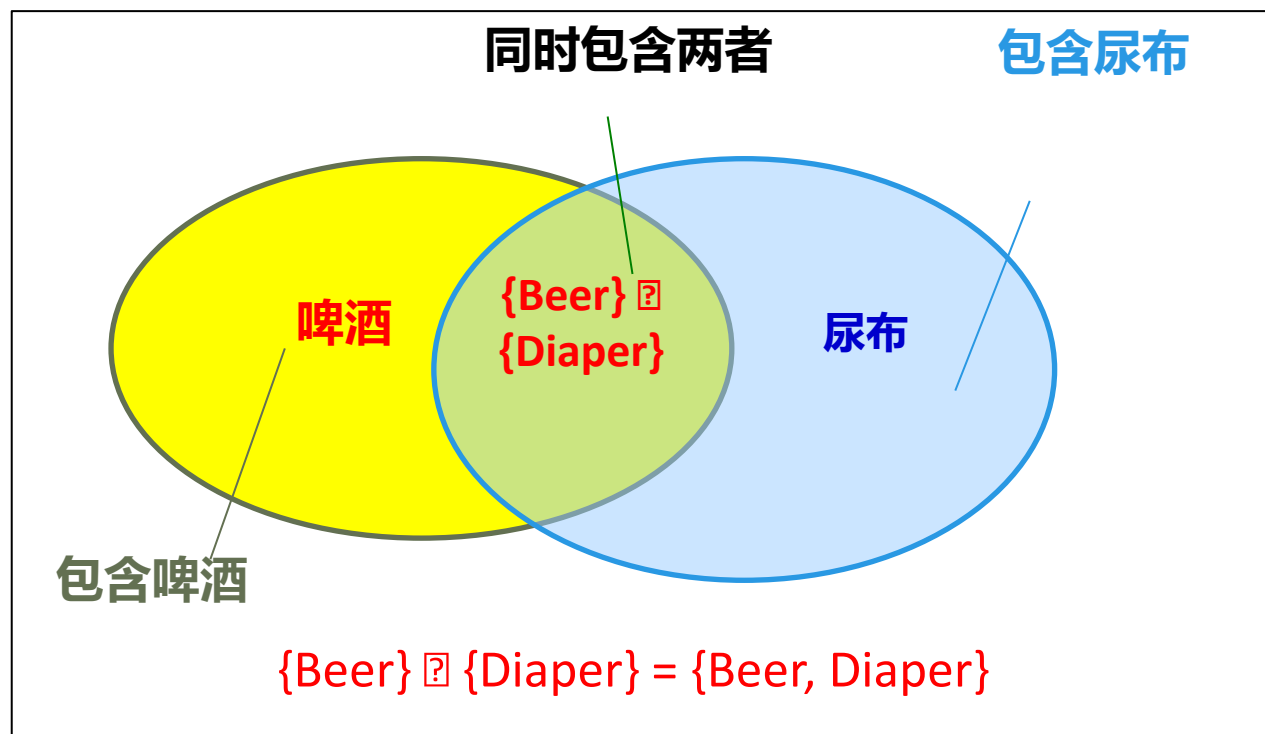
A	B	C	D
2/8	3/8	4/8	6/8

答案：C。

支持度统计的是包含完整项集的事务。

从频繁项集到关联规则

- 与项集相比，关联规则往往更有信息量
 - 关联规则为频繁项集增加了方向与条件含义。
 - 例. Diaper $\Rightarrow$ Beer，包含尿布的事务也很可能包含啤酒。



$$\underbrace{\{Beer\} \cup \{Diaper\}}_{\text{union of items}} = \{Beer, Diaper\}$$
$$\underbrace{T_{Beer} \cap T_{Diaper}}_{\text{intersection of transactions}} = \text{transactions containing both}$$

注：X $\Rightarrow$ Y：两个项集的并集。该集合同时包含 X 与 Y。

关联规则

- 如何计算一条关联规则的强度
 $X \Rightarrow Y$ (X 与 Y 都是项集, 例如 Diaper $\Rightarrow$ Beer) ?

- 先计算下面两个度量 s 与 c 。

- $X \Rightarrow Y$ 的支持度 s

- 例. $s\{\text{Diaper}, \text{Beer}\} = 3/5 = 0.6$ (即 60%)

- $X \Rightarrow Y$ 的置信度 c

- 包含 X 的事务也包含 Y 的条件概率:

$$c = \text{sup}(X, Y) / \text{sup}(X)$$

- 例. $c = \text{sup}\{\text{Diaper}, \text{Beer}\} / \text{sup}\{\text{Diaper}\} = 3/4 = 0.75$

TI D	购买的项
1	啤酒, 坚果, 尿布
2	啤酒, 咖啡, 尿布
3	啤酒, 尿布, 鸡蛋
4	坚果, 鸡蛋, 牛奶
5	坚果, 咖啡, 尿布, 鸡蛋, 牛奶

支持度衡量普遍程度。置信度衡量条件可靠性。任一指标单独都不能确立关联或因果。

规则方向会改变分母

$$\text{conf}(X \Rightarrow Y) = \text{support}(X \cup Y) / \text{support}(X)$$

TID	购买的项
1	啤酒, 坚果, 尿布
2	啤酒, 咖啡, 尿布
3	啤酒, 尿布, 鸡蛋
4	坚果, 鸡蛋, 牛奶
5	坚果, 咖啡, 尿布, 鸡蛋, 牛奶

规则	联合计数	左件计数	置信度
尿布 $\Rightarrow$ 啤酒	3	4	$3/4 = 75\%$
啤酒 $\Rightarrow$ 尿布	3	3	$3/3 = 100\%$

支持度是对称的。置信度是有方向的。

挖掘频繁项集与关联规则

关联规则挖掘

- 给定两个阈值: minsup、minconf
- 找出全部规则 $X \Rightarrow Y (s, c)$, 使得 $s \geq \text{minsup}$ 且 $c \geq \text{minconf}$

设 minsup = 50%

- 频繁 1-项集: Beer: 3, Nuts: 3, Diaper: 4, Eggs: 3
- 频繁 2-项集: {Beer, Diaper}: 3

设 minconf = 50%

- $\text{Beer} \Rightarrow \text{Diaper} (s=60\%, c=100\%)$
- $\text{Diaper} \Rightarrow \text{Beer} (s=60\%, c=75\%)$

TID	购买的项
1	啤酒, 坚果, 尿布
2	啤酒, 咖啡, 尿布
3	啤酒, 尿布, 鸡蛋
4	坚果, 鸡蛋, 牛奶
5	坚果, 咖啡, 尿布, 鸡蛋, 牛奶



观察:

- 挖掘关联规则与挖掘频繁模式是非常接近的问题
- 对大规模数据需要可扩展方法

(问: 满足这两个条件的规则是否已全部列出?)

关联规则挖掘的两个阶段



- ① 支持度剪枝作用于项集。
- ② 置信度过滤作用于有方向的规则。

把模式发现与规则评估分开。

支持度、置信度与解释

度量	公式	问题
规则支持度	$s(X \cup Y)$	该规则覆盖整个数据库的多大比例?
置信度	$s(X \cup Y) / s(X)$	在含 X 的事务中, Y 出现的频率如何?
提升度 (稍后)	$\text{conf}(X \Rightarrow Y) / s(Y)$	置信度与 Y 的基率相比如何?

注: 关联并不意味着因果。

挑战：频繁模式太多了！

- 一个长模式包含组合数量级的子模式
- 下面的 TDB1 含有多少频繁项集 ($\text{minsup} = 1$) ?
 - TDB1 : T1 : {a1, ..., a50}; T2 : {a1, ..., a100}
 - 我们试一试
 - 1-项集: {a1}: 2, {a2}: 2, ..., {a50}: 2, {a51}: 1, ..., {a100}: 1,
 - 2-项集: {a1, a2}: 2, ..., {a1, a50}: 2, {a1, a51}: 1 ..., ..., {a99, a100}: 1,
 - ..., ..., ..., ...
 - 99-项集: {a1, a2, ..., a99}: 1, ..., {a2, a3, ..., a100}: 1
 - 100-项集: {a1, a2, ..., a100}: 1
- 频繁项集的总数:

$$\binom{100}{1} + \binom{100}{2} + \binom{100}{3} + \cdots + \binom{100}{100} = 2^{100} - 1$$

对任何人来说都过大，
无法计算或存储！

组合搜索空间

- 数据库可以很小，但模式空间仍然可以极其庞大。

项数 d	可能的非空项集 $2^d - 1$
10	1,023
20	1,048,575
50	$> 10^{15}$
100	$> 10^{30}$

高效挖掘依赖于紧凑表示与剪枝。

用压缩形式表达模式：闭模式

如何应对这一挑战？

- 方案 1：闭模式：模式（项集） x 是闭的，当且仅当 x 频繁，且不存在超模式 $y \supset x$ 与 x 具有相同支持度
 - 设事务 TDB1： T1: {a1, ..., a50}; T2: {a1, ..., a100}
 - 设 minsup = 1。TDB1 含有多少闭模式？
 - 两个： P1: “{a1, ..., a50}: 2”; P2: “{a1, ..., a100}: 1”
- 闭模式是频繁模式的无损压缩
 - 减少模式数量，但不丢失支持度信息！
 - 你仍可以说：“{a2, ..., a40}: 2”，“{a5, a51}: 1”

用压缩形式表达模式：极大模式

- 方案 2：极大模式：模式 X 是极大模式，当且仅当 X 频繁且不存在频繁超模式 $Y \supset X$
- 与闭模式有何不同？
 - 并不关心极大模式之子模式的真实支持度
 - 设事务库 TDB1: T1: {a1, ..., a50}; T2: {a1, ..., a100}
 - 设 $\text{minsup} = 1$ 。TDB1 含有多少极大模式？
 - 一个: P: "{a1, ..., a100}: 1"
- 极大模式是有损压缩！
 - 我们只知道 {a1, ..., a40} 是频繁的
 - 但不再知道 {a1, ..., a40} 等的真实支持度！
- 在许多应用中，挖掘闭模式比挖掘极大模式更可取

闭频繁模式与极大频繁模式

性质	闭频繁模式	极大频繁模式
定义	没有真超集具有相同支持度	没有真超集是频繁的
保留的信息	可恢复精确支持度	只能恢复是否频繁
压缩程度	通常模式更多	通常模式更少
最适合何时使用	精确支持度值很重要	只需关心频繁边界

每个极大频繁模式都是闭频繁模式， 但并非每个闭频繁模式都是极大的。

Maximal frequent patterns $\subseteq$ Closed frequent patterns $\subseteq$ Frequent patterns

Menti 检查：闭的还是极大的？

问题：X 的支持度为 5。X 没有任何真超集支持度为 5，但至少有一个真超集支持度为 3 且仍频繁。哪项正确？

A	B	C	D
X 仅是极大的	X 是闭的但不是极大的	X 两者都不是	X 的支持度必须是 3

请通过 Menti 提交答案！

答案：B

相等支持度决定闭性。任一频繁超集决定极大性。

高效模式挖掘方法

- 频繁模式的向下封闭性
 - Apriori 算法
 - Apriori 的扩展或改进
- 用垂直数据格式挖掘频繁模式
- FPGrowth: 频繁模式增长方法
- 挖掘闭模式



频繁模式的向下封闭性

- 观察：从 TDB1: T1: {a1, ..., a50}; T2: {a1, ..., a100}
 - 得到频繁项集: {a1, ..., a50}
 - 其子集也都频繁: {a1}, {a2}, ..., {a50}, {a1, a2}, ..., {a1, ..., a49}, ...
 - 频繁模式之间存在隐藏关系!
- 频繁模式的向下封闭性 (也称 “Apriori” 性质)
 - Apriori: 频繁项集的任何子集必定频繁
 - 若 {beer, diaper, nuts} 频繁, 则 {beer, diaper} 也频繁
 - 每个包含 {beer, diaper, nuts} 的事务也包含 {beer, diaper}
- 高效挖掘方法
 - 若项集 s 的任一子集不频繁, 则 s 不可能频繁——何必还考虑 s ! ?

剪枝的利刃!

支持度单调性

If $Y \subset X$, then $\text{support}(Y) \geq \text{support}(X)$

项集	匹配的单车出行	支持度计数
{订阅用户}	T1, T2, T3, T7	4
{订阅用户, 早晨}	T1, T2, T3	3
{订阅用户, 早晨, 中心站}	T1, T2	2

置信度不具有这种反单调性质。

Apriori 剪枝规则



$\{\text{早晨, 中心站}\} \subset \{\text{订阅用户, 早晨, 中心站}\}$

一个不频繁子集就足以否决整个候选。

Apriori 剪枝与可扩展挖掘方法

- Apriori 剪枝原理：若某项集不频繁，则不应再生成其超集！ (Agrawal & Srikant @VLDB'94, Mannila, et al. @ KDD' 94)
- **可扩展挖掘方法：三大途径**
 - 逐层、基于连接的方法：Apriori (Agrawal & Srikant@VLDB'94)
 - 垂直数据格式方法：Eclat (Zaki, Parthasarathy, Ogihara, Li @KDD'97)
 - 频繁模式投影与增长：FPgrowth (Han, Pei, Yin @SIGMOD'00)

Apriori: 候选生成与测试方法

- Apriori 概要（逐层、候选生成与测试）
 - 扫描数据库一次得到频繁 1-项集
 - 重复
 - 由长度为 k 的频繁项集生成长度为 $(k+1)$ 的候选项集
 - 对照数据库测试候选，按 minsup 找出频繁 $(k+1)$ -项集
 - 令 $k := k + 1$
 - 直到无法再生成频繁集或候选集
 - 返回得到的全部频繁项集

Apriori 算法（伪代码）

C_k : Candidate itemset of size k

F_k : Frequent itemset of size k

$K := 1$;

$F_k := \{\text{frequent items}\}$; // frequent 1-itemset

While ($F_k \neq \emptyset$) **do** { // when F_k is non-empty

$C_{k+1} := \text{candidates generated from } F_k$; // candidate generation

 Derive F_{k+1} by counting candidates in C_{k+1} with respect to TDB at minsup;

$k := k + 1$

}

return $\cup_k F_k$ // return F_k generated at each level

Apriori 按广度优先搜索，一次处理一种项集长度。

Apriori 算法——示例

数据库 TDB

TID	项
10	A, C, D
20	B, C, E
30	A, B, C, E
40	B, E

minsup = 2

$C1$

第 1 次扫描

项集	sup
{A}	2
{B}	3
{C}	3
{D}	1
{E}	3

$F1$

项集	sup
{A}	2
{B}	3
{C}	3
{E}	3

连接与剪枝

$C2$

$C2$

$F2$

第 2 次扫描

项集	sup
{A, C}	2
{B, C}	2
{B, E}	3
{C, E}	2

项集	sup
{A, B}	1
{A, C}	2
{A, E}	1
{B, C}	2
{B, E}	3
{C, E}	2

项集
{A, B}
{A, C}
{A, E}
{B, C}
{B, E}
{C, E}

连接与剪枝

$C3$

第 3 次扫描

$F3$

项集
{B, C, E}

项集	sup
{B, C, E}	2

Apriori: 实现技巧

□ 如何生成候选?

□ 步骤 1: 自连接 F_k

□ 步骤 2: 剪枝

□ 候选生成示例

□ $F_3 = \{abc, abd, acd, ace, bcd\}$

□ **自连接: $F_3 * F_3$**

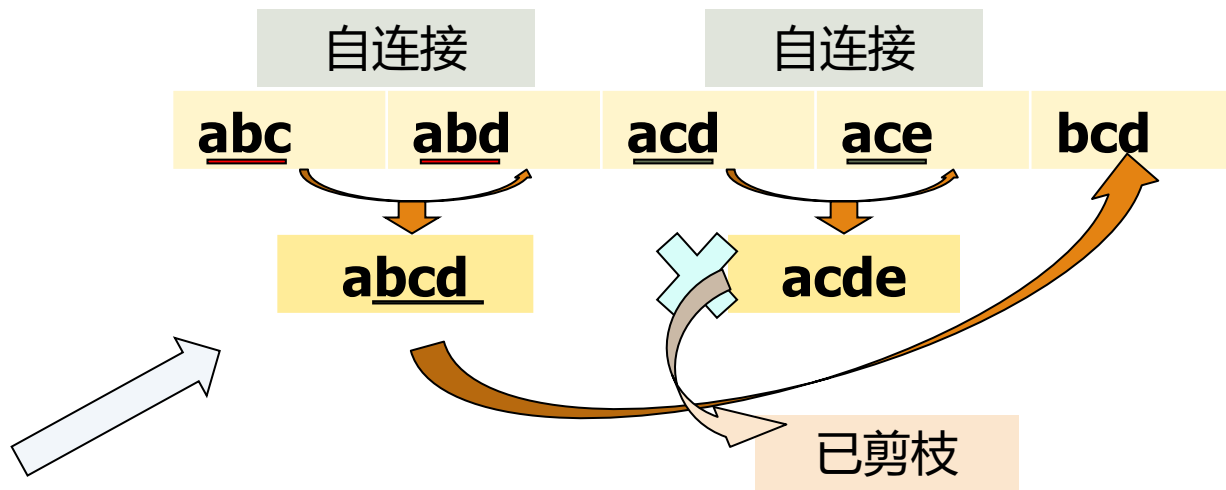
□ $abcd$ 来自 abc 与 abd

□ $acde$ 来自 acd 与 ace

□ **剪枝:**

□ $acde$ 被去掉, 因为 ade 不在 F_3 中

□ $C_4 = \{abcd\}$



自连接并不意味着连接 F_k 中的每一对。仅当两个项集的前 $(k-1)$ 个有序项相同时才连接。这样每个候选只生成一次，避免重复。

Apriori: 优势与局限

优势

- ✓ 简单的逐层搜索
- ✓ 在固定 minsup 下完备
- ✓ 剪枝逻辑透明
- ✓ 对许多稀疏数据效果好

VS

局限

- 候选集很大
- 反复扫描数据库
- 中间计数占用内存高
- 稠密数据或低 minsup 时较慢

第1部分小结与第2部分问题

现在能做的

- ✓ 定义事务与项集
- ✓ 计算支持度与置信度
- ✓ 推演 Apriori 的连接与剪枝
- ✓ 解释闭模式与极大模式

仍待解决的

- 避免反复扫描
- 避免巨大候选表
- 压缩重复的事务前缀
- 在置信度之外评估规则

第2部分的问题:

不同的表示能否让挖掘更快、输出更有用?

附录

第1部分演算示例

Apriori 演练：数据与阈值

TID	项
T1	订阅用户, 早晨, 中心站, 短时
T2	订阅用户, 早晨, 中心站, 短时
T3	订阅用户, 早晨, 河滨站, 短时
T4	临时用户, 周末, 河滨站, 长时
T5	临时用户, 周末, 博物馆站, 长时
T6	临时用户, 周末, 河滨站, 长时
T7	订阅用户, 傍晚, 中心站, 短时
T8	临时用户, 周末, 博物馆站, 长时

使用 minsup 计数 = 3。

第 1 遍: C₁ 与 L₁

项	计数	保留?	项	计数	保留?
订阅用户	4	是	临时用户	4	是
短时	4	是	长时	4	是
早晨	3	是	周末	4	是
中心站	3	是	河滨站	3	是
博物馆站	2	否	傍晚	1	否

只有频繁单项才能参与更大的频繁项集。

第 2 遍：候选对 C₂

代表性候选	计数	保留?
{订阅用户, 短时}	4	是
{订阅用户, 早晨}	3	是
{早晨, 中心站}	2	否
{临时用户, 周末}	4	是
{临时用户, 河滨站}	2	否
{周末, 长时}	4	是

计数把 C₂ 变成频繁集 L₂。

频繁对 L2

订阅用户一侧的对

- {订阅用户, 短时}: 4
- {订阅用户, 早晨}: 3
- {订阅用户, 中心站}: 3
- {早晨, 短时}: 3
- {中心站, 短时}: 3

VS

临时用户一侧的对

- {临时用户, 周末}: 4
- {临时用户, 长时}: 4
- {周末, 长时}: 4
- 河滨站没有频繁对

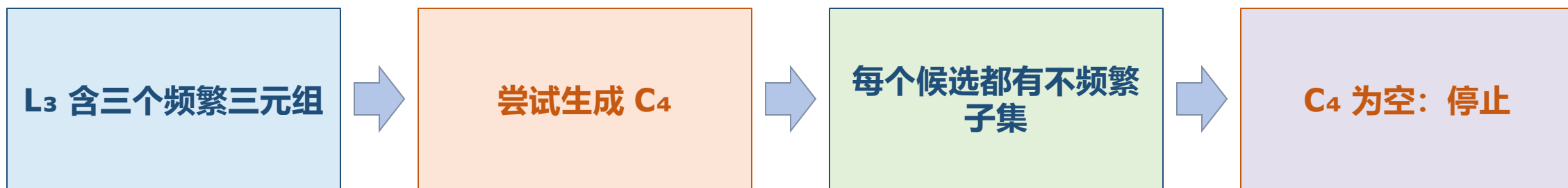
频繁单项不会自动产生频繁对。

连接并剪枝候选三元组

候选	所需 2-子集	判定
{订阅用户, 早晨, 短时}	三个对都在 L_2 中	生成并计数
{订阅用户, 早晨, 中心站}	{早晨, 中心站} 不在 L_2 中	剪枝
{临时用户, 周末, 长时}	三个对都在 L_2 中	生成并计数

子集测试在支持度计数前去掉不可能的三元组。

停止条件与完备性



$$\text{频繁输出} = L_1 \cup L_2 \cup L_3$$

向下封闭性保证 Apriori 不会剪掉真正频繁的项集。

由频繁项集生成规则

$F = \{\text{临时用户, 周末, 长时}\}$, 支持度 = 4/8

左件	右件	置信度
临时用户	周末, 长时	$4/4 = 1.00$
周末	临时用户, 长时	$4/4 = 1.00$
长时	临时用户, 周末	$4/4 = 1.00$
临时用户, 周末	长时	$4/4 = 1.00$

minsup 控制项集发现。minconf 控制规则输出。

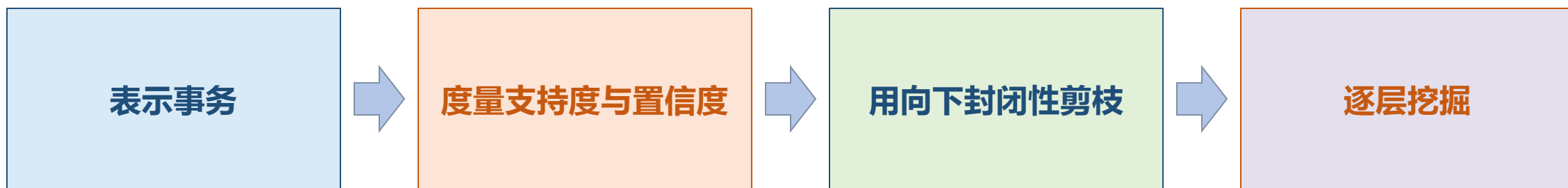
快速检查: Apriori

问题: 当 $\{A, B\}$ 不频繁时, Apriori 为何可以剪掉 $\{A, B, C\}$?

A	B	C	D
置信度下降	加入 C 后支持度不可能上升	提升度变为零	数据库是稀疏的

答案: B。超集的支持度不可能高于其子集。

第1部分知识地图



核心思想是避免计数不可能频繁的候选。

推荐阅读（基本概念）

- ❑ R. Agrawal, T. Imielinski, and A. Swami, “Mining association rules between sets of items in large databases”, in Proc. of SIGMOD'93
- ❑ R. J. Bayardo, “Efficiently mining long patterns from databases”, in Proc. of SIGMOD'98
- ❑ N. Pasquier, Y. Bastide, R. Taouil, and L. Lakhal, “Discovering frequent closed itemsets for association rules”, in Proc. of ICDT'99
- ❑ J. Han, H. Cheng, D. Xin, and X. Yan, “Frequent Pattern Mining: Current Status and Future Directions”, Data Mining and Knowledge Discovery, 15(1): 55-86, 2007

第2部分：挖掘、压缩与评估

挖掘	压缩	评估
ECLAT 与 FP-Growth	闭项集与极大项集	基率、提升度、 χ^2 与零不变度量

目标是更快挖掘、更小输出、更好解释。

回顾检查：向下封闭性

哪一陈述支持在计数前剪掉 $\{A, B, C, D\}$?

A	B	C	D
$\{A, B\}$ 是频繁的	$\{A, B, C\}$ 是不频繁的	A 的置信度很高	D 的提升度很高

答案：B。

任一不频繁子集都会使每个超集不频繁。

Apriori 算法（再次给出伪代码）

C_k : Candidate itemset of size k

F_k : Frequent itemset of size k

$K := 1$;

$F_k := \{\text{frequent items}\}$; // frequent 1-itemset

While ($F_k \neq \emptyset$) **do** { // when F_k is non-empty

$C_{k+1} := \text{candidates generated from } F_k$; // candidate generation

 Derive F_{k+1} by counting candidates in C_{k+1} with respect to TDB at minsup;

$k := k + 1$

}

return $\cup_k F_k$ // return F_k generated at each level

Apriori 按广度优先搜索，一次处理一种项集长度。

Apriori: 改进与替代方法

□ 减少事务数据库扫描遍数

- 划分 (如 Savasere, et al., 1995)
- 动态项集计数 (Brin, et al., 1997)

后续幻灯片将讨论

□ 缩小候选数量

- 散列 (如 DHP: Park, et al., 1995)
- 用支持度下界剪枝 (如 Bayardo 1998)
- 抽样 (如 Toivonen, 1996)

后续幻灯片将讨论

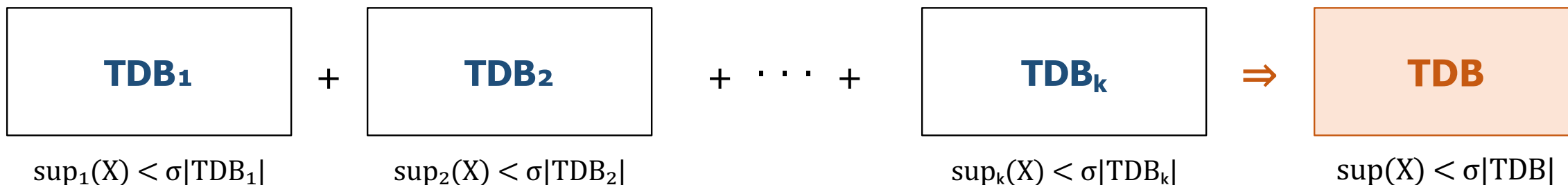
□ 探索特殊数据结构

- 树投影 (Agarwal, et al., 2001)
- H-miner (Pei, et al., 2001)
- 超立方体分解 (如 LCM: Uno, et al., 2004)

划分法：数据库只扫描两遍

□ **定理：TDB 中任何全局频繁项集必在至少一个划分中频繁。**

σ 表示最小相对支持度阈值。



定理为何成立

假设 X 在每个划分中都不频繁：

$$\text{sup}_i(X) < \sigma |TDB_i| \quad \text{for every } i$$

把局部支持度计数相加：

$$\text{sup}_D(X) = \sum_i \text{sup}_i(X) < \sigma \sum_i |TDB_i| = \sigma |TDB|$$

因此 X 全局不频繁。逆否命题即证明该定理。

局部频繁集的并集包含每一个全局频繁项集。

划分法：数据库只扫描两遍

扫描 1 局部挖掘

- 1 划分数据库**
选择能放入主存的分块。
- 2 挖掘每个划分**
用阈值 $\sigma|TDB_i|$ 求局部频繁集 L_i 。
- 3 形成候选并集**
 $C = L_1 \cup L_2 \cup \dots \cup L_k$

为何要能放入内存？局部挖掘可用若干次内存遍数，而不再做整盘扫描。

扫描 2 全局验证

- 1 扫描完整 TDB**
再处理每一条事务一次。
- 2 统计精确全局支持度**
只对候选 $X \in C$ 计数 $\text{sup}_D(X)$ 。
- 3 施加全局阈值**
 $F = \{X \in C : \text{sup}_D(X) \geq \sigma|TDB|\}$

C 可能含假阳性。定理保证 C 没有假阴性。

**该方法做两次完整磁盘扫描。
局部挖掘可以使用多次内存遍数。**

直接散列与剪枝 (**DHP**)

□ **第一遍扫描：对每条事务计数 1-项集，并对每个 2-项集散列。**

□ **对每条事务 T：**

1. 枚举 T 中每个 2-项集 X
2. $b = \text{hash}(X)$
3. $\text{bucketCount}[b] = \text{bucketCount}[b] + 1$

示例：

$T1 = \{a, b, d\}$ 含 $\{ab, ad, bd\}$

$h(ab) = h(ad) = V1$ ，而 $h(bd) = V2$

T1 给 V1 加 2、给 V2 加 1。

$T2 = \{c, e\}$ ， $h(ce) = V1$ ，故 T2 给 V1 加 1。

完整第一遍扫描之后

桶	映射到此桶的对的例子	计数
V1	{ab, ad, ce, ...}	35
V2	{bd, be, de, ...}	298
...	...	...
Vn	{yz, qs, wt, ...}	58

桶计数 = 映射到该桶的全部 2-项集出现次数之和。

注：不同项集可能散列到同一桶。

直接散列与剪枝 (**DHP**)

- 第一遍扫描后，将各桶计数与 $\text{minsup}(\sigma)$ 比较。

$$\text{support}(X) \leq \text{bucketCount}[\text{hash}(X)]$$

该桶包含 x 及所有碰撞项集，因此其计数是 $\text{support}(X)$ 的上界。

$$\text{minsup}(\sigma) = 80$$

σ 表示最小支持度计数。

- 剩余的项集仍需在第二遍扫描中精确计数支持度。
- 推广到 k -项集。

桶	映射的项集	计数	判定
V1	{ab, ad, ce, ...}	35	全部剪掉
V2	{bd, be, de, ...}	298	保留为候选
Vn	{yz, qs, wt, ...}	58	全部剪掉

桶计数低 ($< \text{minsup}$) 即可保证不频繁。

桶计数高 ($\geq \text{minsup}$) 只说明有可能。

Apriori 的改进与替代方法

目标	方法	主要思想
减少数据库扫描	划分法、动态项集计数	每遍计数更多候选，或只验证缩小后的并集
减少候选	DHP、支持度下界、抽样	在精确计数前拒绝或近似候选
改变搜索结构	树投影、H-Miner、LCM	使用紧凑结构与专门遍历

所有方法保持或近似同一频繁项集目标；它们降低的是不同代价

。

探索垂直数据格式: ECLAT

- ECLAT (等价类变换) : 用集合交运算的深度优先搜索算法 [Zaki et al. @KDD'97]

- 垂直格式: $t(X) = \{\text{包含 } X \text{ 的事务 ID}\}$

- Tid-List 的性质

- $t(X) = t(Y)$: X 与 Y 总是一起出现 (如 $t(ac) = t(d)$)
- $t(X) \subseteq t(Y)$: 含 X 的事务总含 Y (如 $t(ac) \subseteq t(ce)$)

- 频繁模式: 垂直交运算

- 用 diffset 加速挖掘

- 只存储相对父 Tid-list 被去掉的 TID。

- $t(e) = \{T10, T20, T30\}$, $t(ce) = \{T10, T30\} \rightarrow \text{Diffset}(ce, e) = \{T20\}$

水平数据格式的事务数据库

TID	项集
10	a, c, d, e
20	a, b, e
30	b, c, e

垂直数据格式的事务数据库

项	TidList
a	10, 20
b	20, 30
c	10, 30
d	10
e	10, 20, 30

$t(e) = \{T10, T20, T30\};$
 $t(a) = \{T10, T20\};$
 $t(ae) = \{T10, T20\}$

用 TID 集合交运算求 ECLAT 支持度

项	TID 集合
订阅用户	{T1, T2, T3, T7}
早晨	{T1, T2, T3}
订阅用户 $\cup$ 早晨	{T1, T2, T3}

$$t(\text{Subscriber} \cup \text{Morning}) = t(\text{Subscriber}) \cap t(\text{Morning})$$

交集含三个 TID，因此支持度计数为 3。

ECLAT 搜索空间：等价类

- 固定项序： $a < b < c < d$ 。只用更后的项扩展每个前缀。

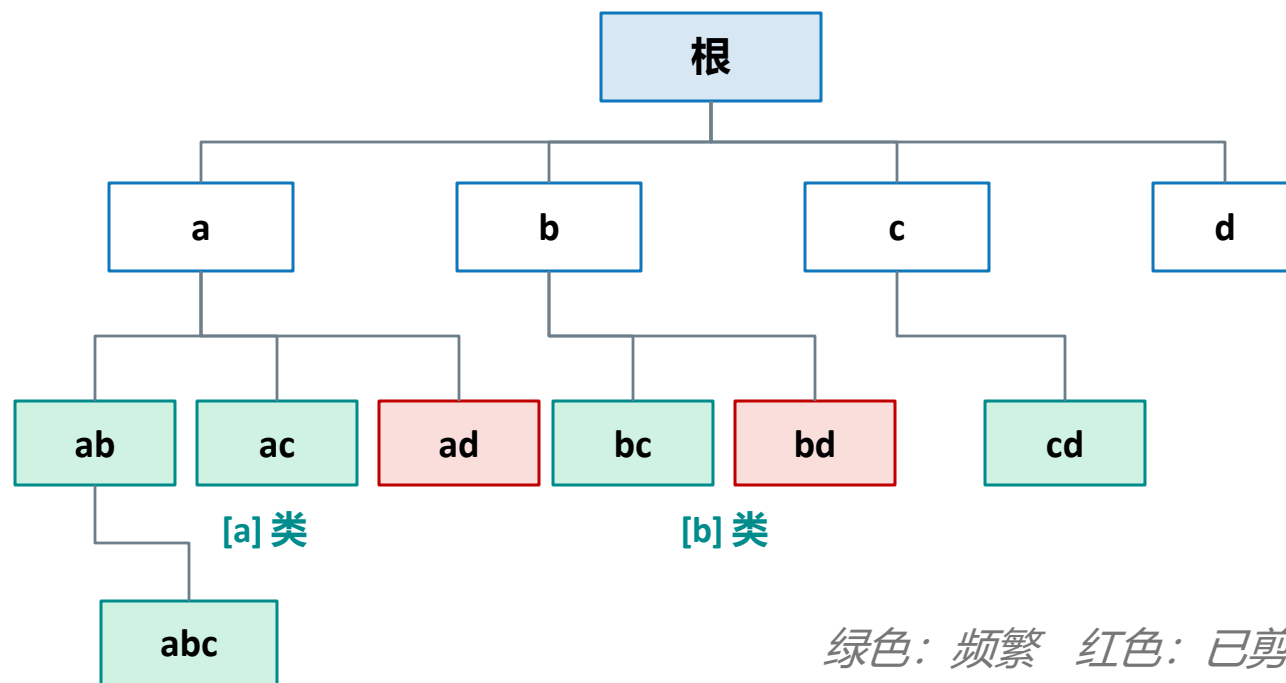
示例数据库 (minsup = 2)

T1: {a,b,c} T2: {a,c,d}

T3: {b,c,d} T4: {a,b,c}

项	TID-list	支持度
a	{T1, T2, T4}	3
b	{T1, T3, T4}	3
c	{T1, T2, T3, T4}	4
d	{T2, T3}	2

基于前缀的搜索空间



相同前缀的项集构成一个等价类；ECLAT 按深度优先探索一个类。

ECLAT: 深度优先的模式扩展

□ 核心操作：扩展前缀，并对其扩展的 TID-list 求交。

递归过程

ECLAT(P, E)

1. for each extension X_i in E :
2. $Q = P \text{ union } \{X_i\}$
3. output Q
4. for each X_j after X_i :
5. $T(Q \text{ union } X_j) = T(Q)$
 intersect $T(X_j)$
6. if support $\geq$ minsup:
 keep the extension
7. recursively mine the extensions of Q

演算示例：前缀 a

模式	TID-list	支持度	判定
ab	{T1, T4}	2	频繁
ac	{T1, T2, T4}	3	频繁
ad	{T2}	1	剪枝
abc	$t(ab) \text{ intersect } t(ac) = \{T1, T4\}$	2	此子集

因为 ad 不频繁，扩展 abd 与 acd 被剪枝。

频繁项集：{a}, {b}, {c}, {d}, {ab}, {ac}, {bc}, {cd}, 以及 {abc}

ECLAT 差集与实际权衡

- ❑ ECLAT 用 TID-list 交运算替代反复扫描数据库。
- ❑ Diffset 只存储父（前缀）与其扩展之间变化的部分。

$$\text{Diffset: } \Delta(e \rightarrow ce) = t(e) - t(ce)$$

集合	TIDs
$t(e)$	{T10, T20, T30}
$t(ce)$	{T10, T30}
Δ	{T20}

何时有帮助

- TID 集合能放入内存
- 位集或交运算很快
- 深度优先搜索限制活动状态

主要风险

很长的 Tid 列表以及真正巨大的模式空间

Apriori 与 ECLAT

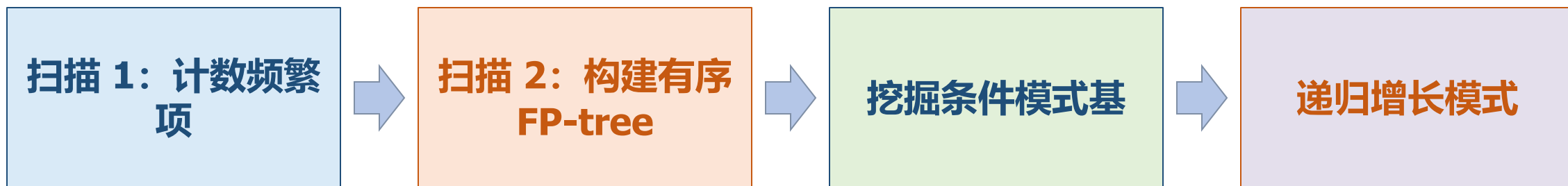
维度	Apriori	ECLAT
数据格式	水平事务	垂直 TID 集合
搜索	广度优先	通常深度优先
核心操作	生成并计数	TID 集合求交
典型瓶颈	候选与反复扫描	交运算内存与很长的 TID 列表

表示方式改变瓶颈；
它并不能消除组合增长。

为何用模式增长挖掘频繁模式？

- Apriori：广度优先搜索挖掘算法
 - 先找出完整的频繁 k -项集
 - 再导出频繁 $(k+1)$ -项集候选
 - 再扫描数据库找出真正的频繁 $(k+1)$ -项集
- 动机：能否设计深度优先搜索挖掘算法？
 - 对频繁项集 ρ ，后续搜索能否限制在只含 ρ 的那些事务中？
- 这一思路导向频繁模式增长方法：FPGrowth

FP-Growth 概览



FP-Growth 避免全局候选层 C_k 。

示例：从事务数据库到有序频繁项表

示例：一个事务数据库样本

TID	事务中的项
100	{f, a, c, d, g, i, m, p}
200	{a, b, c, f, l, m, o}
300	{b, f, h, j, o, w}
400	{b, c, k, s, p}
500	{a, f, c, e, l, p, m, n}

设 $\text{min_support} = 3$

□ 扫描数据库一次，找出单项频繁模式：

f:4, a:3, c:4, b:3, m:3, p:3

□ 按频率降序排列频繁项：f-list

F-list = f-c-a-b-m-p

□ 再扫描数据库，用每条事务的有序频繁项表构造 FP-tree

TID	事务中的项	有序频繁项表
100	{f, a, c, d, g, i, m, p}	<i>f, c, a, m, p</i>
200	{a, b, c, f, l, m, o}	<i>f, c, a, b, m</i>
300	{b, f, h, j, o, w}	<i>f, b</i>
400	{b, c, k, s, p}	<i>c, b, p</i>
500	{a, f, c, e, l, p, m, n}	<i>f, c, a, m, p</i>

为何 FP-Growth 使用一个全局次序

单项频繁模式

f:4, a:3, c:4, b:3, m:3, p:3

第二次扫描数据库时，对
每条事务去掉不频繁项

对频繁项排序

F-list = f-c-a-b-m-p

TID	事务中的原始项
100	{f, a, c, d, g, i, m, p}
200	{a, b, c, f, l, m, o}
300	{b, f, h, j, o, w}
400	{b, c, k, s, p}
500	{a, f, c, e, l, p, m, n}

一致的次序把常见项组合变成共享前缀。

示例：由事务数据库构造 FP-tree

TID	有序频繁项表
100	<i>f, c, a, m, p</i>
200	<i>f, c, a, b, m</i>
300	<i>f, b</i>
400	<i>c, b, p</i>
500	<i>f, c, a, m, p</i>

插入第 1 个频繁项表之后：
“f, c, a, m, p”

项	频率	头表
f	4	
c	4	
a	3	
b	3	
m	3	
p	3	

头表

插入第 2 个频繁项表之后
“f, c, a, b, m”

项	头表
f	
c	
a	
b	
m	
p	

插入全部频繁项表之后

项	头表
f	
c	
a	
b	
m	
p	

FP-Tree 构造：

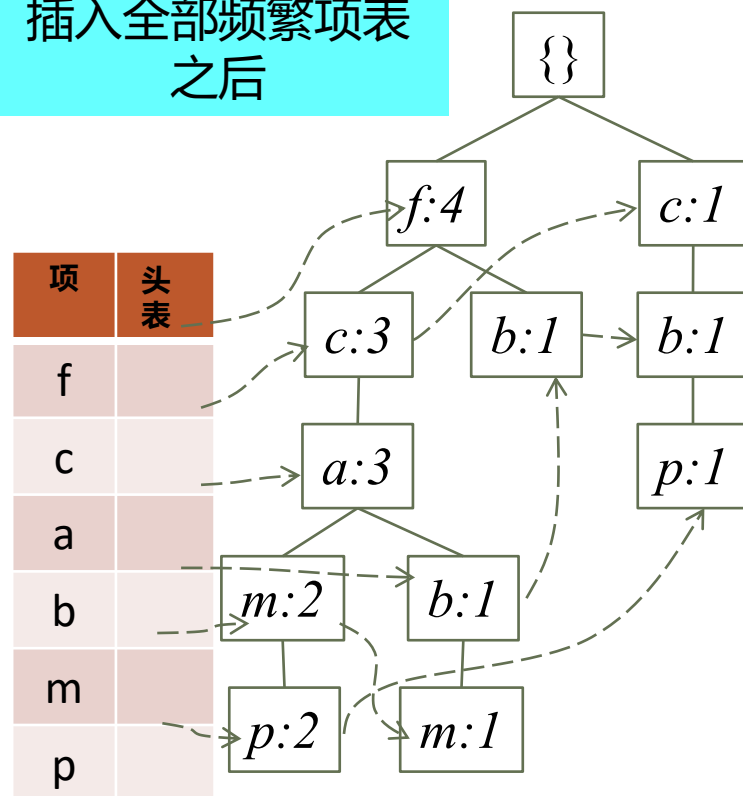
- ① 对每条事务，将其有序频繁项表插入 FP-tree。
- ② 复用公共前缀，增加其结点计数，并为剩余后缀创建新结点。

F-list = f-c-a-b-m-p

f:4, a:3, c:4, b:3, m:3, p:3

阅读已完成的 FP-Tree

插入全部频繁项表
之后



树元素	含义
路径 $f:4 \rightarrow c:3 \rightarrow a:3$	三条事务共享前缀 f-c-a; 四条以 f 开头
两个不同的 b 结点	b 跟在不同前缀之后, 因此结点不能合并
结点计数	到达完整前缀的事务数
头表链接	在条件挖掘时连接同一项的所有结点

去掉不频繁项后, FP-tree 是频繁模式挖掘的无损索引。

挖掘 **FP-Tree**: 基于模式与数据的分治

□ 模式挖掘可按当前模式划分

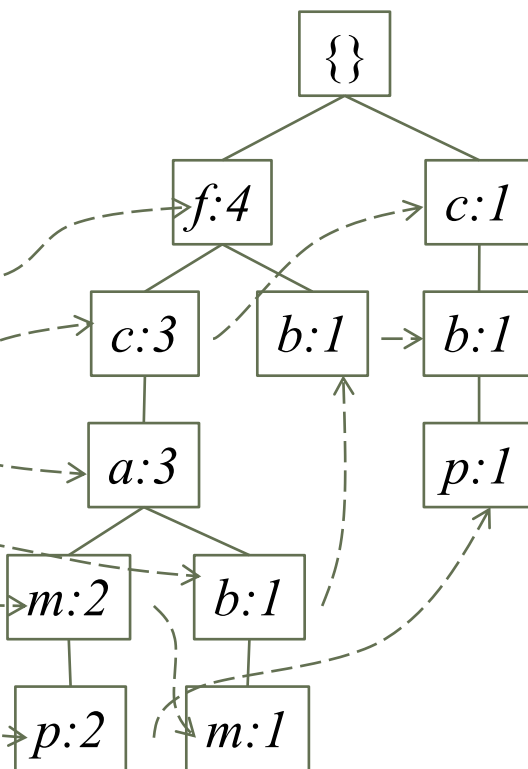
□ p 的条件数据库 (即在 p 存在条件下的数据库): 项 p 的变换前缀路径 (把 p 固定为后缀)

□ 对后缀 p , 收集所有止于 p 的前缀路径:

p 的条件模式基 = $\{fcam:2, cb:1\}$

min_support = 3

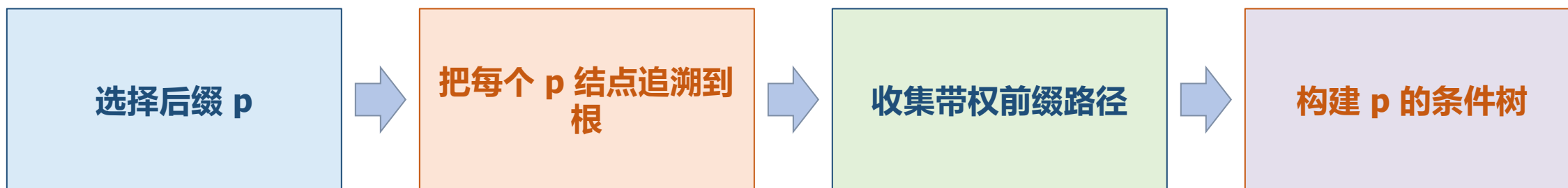
项	频率	头表
f	4	
c	4	
a	3	
b	3	
m	3	
p	3	



各后缀项的条件模式基

项	条件数据库
c	f:3
a	fc:3
b	fca:1, f:1, c:1
m	fca:2, fcab:1
p	fcam:2, cb:1

条件模式基



p 之前的前缀路径	路径计数
f, c, a, m	2
c, b	1

条件挖掘把一个全局问题替换为更小的投影问题。

递归挖掘每个条件数据库

min_support = 3

项	条件数据库
c	f:3
a	fc:3
b	fca:1, f:1, c:1
m	fca:2, fcab:1
p	fcam:2, cb:1

□ 对每个条件数据库

□ 挖掘单项模式

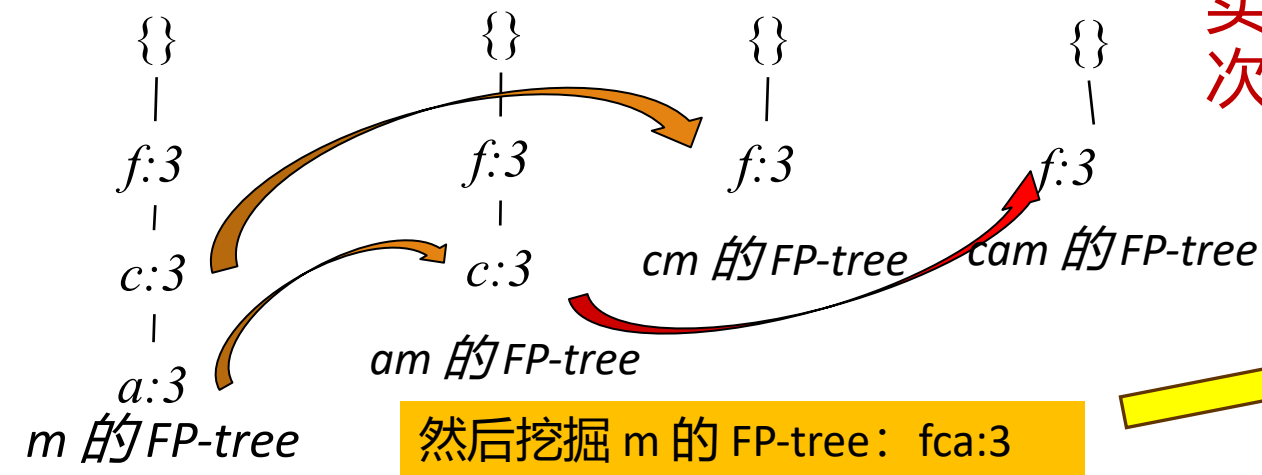
□ 构造其条件 FP-tree，并带着当前后缀递归挖掘模式

p 的条件数据库: $fcam:2, cb:1 \rightarrow c:3$

m 的条件数据库: $fca:2, fcab:1 \rightarrow fca:3$

b 的条件数据库: $fca:1, f:1, c:1 \rightarrow \phi$

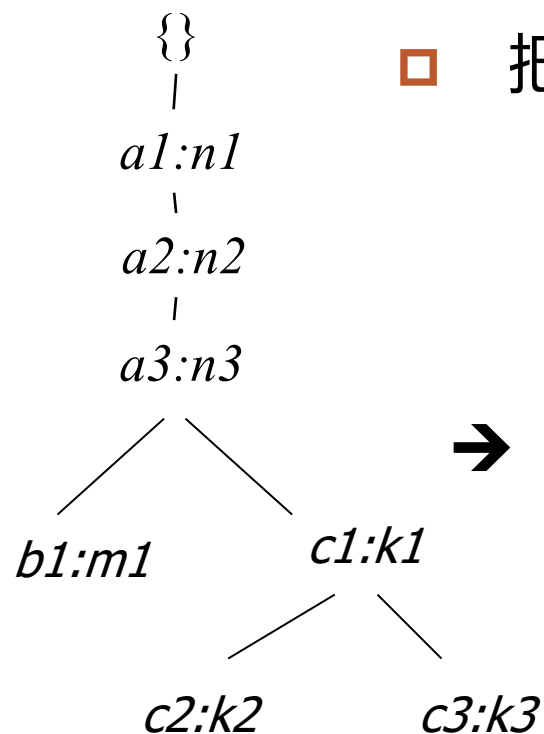
实际上，对单分支 FP-tree，全部频繁模式可一次生成



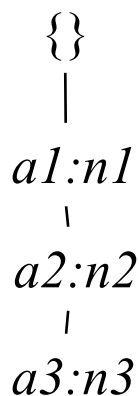
m: 3
fm: 3, cm: 3, am: 3
fcm: 3, fam: 3, cam: 3
fcam: 3

特例：FP-tree 中的单前缀路径

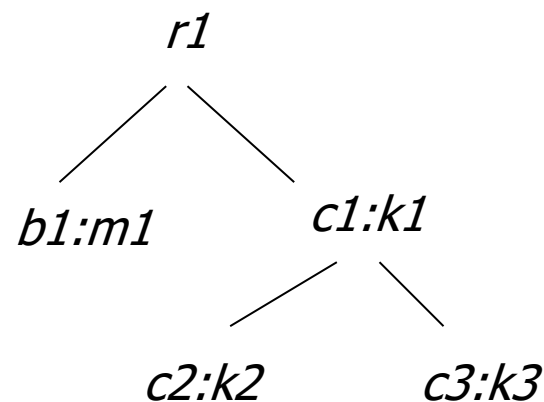
- 设（条件）FP-tree T 有一条共享的单前缀路径 P
- 挖掘可分解为两部分
 - 把单前缀路径压缩为一个结点
 - 把两部分挖掘结果拼接



$r1 =$



+



prefix-only patterns + prefix subsets concatenated with residual patterns

FPGrowth：用模式增长挖掘频繁模式

□ 频繁模式增长（FPGrowth）方法的本质

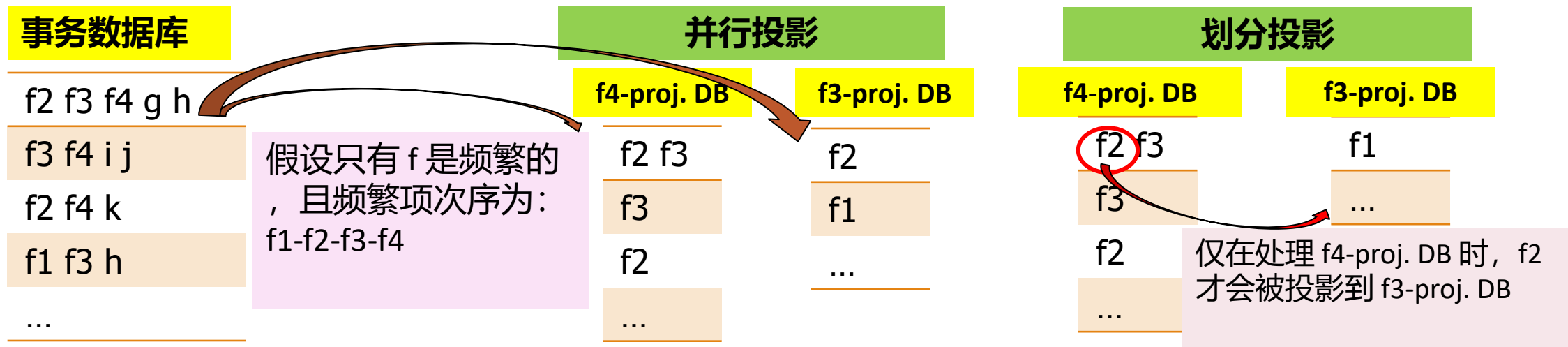
- 找出频繁单项，并按每个这样的单项模式划分数据库
- 对每个划分后的数据库（也称该模式的条件数据库）递归地按上述方式增长频繁模式
- 为便于高效处理，可构造高效数据结构 FP-tree

□ 挖掘变为

- 递归构造并挖掘（条件）FP-tree
- 直到所得 FP-tree 为空，或只含一条路径——单路径会生成其所有子路径组合，每一个都是频繁模式

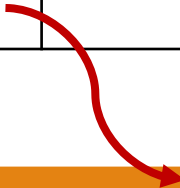
用基于项的数据投影扩展 FP-growth 规模

- 若 FP-tree 放不进内存怎么办？——不要构造全局 FP-tree
 - 按频繁单项“投影”数据库
 - 对每个投影数据库构造并挖掘 FP-tree
- 并行投影 vs. 划分投影
 - 并行投影：对每个频繁项投影数据库
 - 空间代价高，所有划分可并行处理
 - 划分投影：按次序划分数据库
 - 把未处理部分传给后续划分



三类频繁项集挖掘方法族

维度	Apriori	ECLAT	FP-Growth
表示	水平	垂直 TID 集合	FP-tree
搜索	广度优先	通常深度优先	模式增长
主要操作	生成并测试	集合交	条件投影
何时往往有效	稀疏数据且 minsup 适中	TID 集合能放入内存	有序前缀压缩得好



算法选择取决于表示方式与瓶颈。

快速检查：FP-Tree 压缩

问题：为何重复事务能在 FP-tree 中被压缩？

A	B	C	D
项遵循一个全局次序	置信度是反单调的	树只存储规则	每个结点有唯一的项

答案：A。

只有次序一致时，公共前缀才会对齐。

频繁模式的输出也可能过大

紧凑的 FP-tree 仍可表示指数级数量的有效模式。

紧凑挖掘结构

- 共享前缀压缩事务
- FP-growth 避免候选生成
- 树或许能轻松放入内存

有效输出仍然很大

- 每个频繁模式都可能需要输出
- 长事务产生许多子集
- 低 min_sup 进一步放大结果

若 d 个项总是一起出现： 频繁项集数 = $2^d - 1$

闭模式与极大模式在保留不同信息的同时缩小报告集合。

闭 vs. 极大：快速回顾

二者都概括频繁模式，但保留的信息不同。

维度	闭频繁项集	极大频繁项集
定义	没有真超集具有相同支持度	没有真超集是频繁的
保留的信息	频率与支持度信息	仅频率边界
压缩程度	中等	强

每个极大频繁项集都是闭的，但闭项集不必是极大的。

相等支持度决定闭性。任一频繁超集决定极大性。

从闭模式恢复支持度

- 闭模式提供频繁项集及其支持度的无损压缩。

事务	项
T1	{a, b, c}
T2	{a, b, c}
T3	{a, b}

minsup = 2

闭模式	支持度
{a, b}	3
{a, b, c}	2

只存储两个闭模式

对任意频繁项集 X :

$$\text{support}(X) = \max\{ \text{support}(Y) : Y \text{ 是闭的且 } X \subseteq Y \}$$

查询 {a}

$$\max\{\text{support}(ab), \text{support}(abc)\} = \max\{3, 2\} = 3$$

查询 {c}

$$\text{support}(abc) = 2$$

X 的闭包是与 X 有相同支持度的最大超集。

CLOSET+: 直接挖掘闭项集

□ 闭包规则

$$\text{support}(X \cup \{i\}) = \text{support}(X)$$

项 i 属于 $\text{closure}(X)$

1. 投影

从前缀 X 及其投影数据库开始

2. 合并与剪枝

把相同支持度的项并入
 $\text{closure}(X)$
跳过冗余搜索分支

3. 输出

仅当没有更大模式具有相同支持度时才报告 $\text{closure}(X)$

选择模式输出

输出	全部频繁	闭	极大
保留内容	每一个频繁模式	模式及全部支持度	频率边界
何时使用	需要完整枚举	支持度查询很重要	紧凑摘要已足够

注：输出压缩是任务决策，而不只是实现选择。

如何评估规则/模式是否有趣？

- 模式挖掘会生成大量模式/规则
 - 并非所有生成的模式/规则都有趣
- 趣味性度量：客观 vs. 主观
 - 客观趣味性度量：支持度、置信度、相关性、...
 - 主观趣味性度量：不同用户对趣味性的判断可能不同
 - 基于查询：与用户的特定需求相关
 - 对照知识库判断：意外性、新鲜度、时效性

支持度-置信度框架的局限

- 关联规则 “A $\Rightarrow$ B” [s, c] 中的 s 与 c 是否有趣？

请注意！

- 示例：假设某校关于打篮球和/或吃麦片的学生人数有如下统计：

二维列联表

	打篮球	不打篮球	行合计
吃麦片	400	350	750
不吃麦片	200	50	250
列合计	600	400	1000

- 关联规则挖掘可能生成如下规则：

- $play\text{-}basketball \Rightarrow eat\text{-}cereal$ [$400/1000=40\%$, $400/600=66.7\%$] (更高的 s 与 c)

- 但这条强关联规则具有误导性：吃麦片学生的总体比例是 $75\% > 66.7\%$ ，更有信息量的规则是：

- $\neg play\text{-}basketball \Rightarrow eat\text{-}cereal$ [$350/1000=35\%$, $350/400=87.5\%$] (高 s 与 c)

趣味性度量：提升度

- 度量事件是否依赖/相关：提升度

$$lift(B, C) = \frac{c(B \rightarrow C)}{s(C)} = \frac{s(B \cup C)}{s(B) \times s(C)}$$

提升度比 s 与 c 更有信息量

- Lift(B, C) 可以说明 B 与 C 如何相关

- Lift(B, C) = 1: B 与 C 独立

- > 1: 正相关

- < 1: 负相关

- 对本例,

$$lift(B, C) = \frac{400/1000}{600/1000 \times 750/1000} = 0.89$$

$$lift(B, \neg C) = \frac{200/1000}{600/1000 \times 250/1000} = 1.33$$

- 因此 B 与 C 负相关, 因为 $lift(B, C) < 1$; B 与 $\neg C$ 正相关, 因为 $lift(B, \neg C) > 1$

	B	$\neg B$	行合计
C	400	350	750
$\neg C$	200	50	250
列合计	600	400	1000

趣味性度量： χ^2

- 检验相关事件的另一度量： χ^2

$$\chi^2 = \sum \frac{(Observed - Expected)^2}{Expected}$$

- 对右侧表格，

$$\chi^2 = \frac{(400 - 450)^2}{450} + \frac{(350 - 300)^2}{300} + \frac{(200 - 150)^2}{150} + \frac{(50 - 100)^2}{100} = 55.56$$

- 查阅 χ^2 分布表 $\rightarrow$ B、C 相关
- χ^2 检验表明 B 与 C 负相关，因为期望值为 **450**，而观测值仅为 **400**
- 因此 χ^2 也比支持度-置信度框架更有信息量

	B	$\neg B$	行合计
C	400 (450)	350 (300)	750
$\neg C$	200 (150)	50 (100)	250
列合计	600	400	1000

期望值

观测值

提升度与 χ^2 : 它们总是好度量吗?

- ❑ 零事务: 既不含 B 也不含 C 的事务
- ❑ 考察新数据集 D
 - ❑ BC (100) 远少于 $B\bar{C}$ (1000) 与 $\bar{B}C$ (1000), 但有大量 $\bar{B}\bar{C}$ (100000)
 - ❑ B 与 C 不太可能一起发生!
- ❑ 但是, $Lift(B, C) = 8.44 \gg 1$ (提升度显示 B 与 C 强正相关!)
- ❑ $\chi^2 = 670$: $Observed(BC) \gg$ 期望值 (11.85)
- ❑ 过多零事务可能“坏了一锅汤”!

	B	$\bar{B}$	行合计
C	100	1000	1100
$\bar{C}$	1000	100000	101000
列合计	1100	101000	102100

零事务

附上期望值的列联表			
	B	$\bar{B}$	行合计
C	100 (11.85)	1000	1100
$\bar{C}$	1000 (988.15)	100000	101000
列合计	1100	101000	102100

稀疏数据中的零事务

□ 零事务既不含项 A 也不含项 B。

	B	非 B	合计
A	a	b	$a + b$
非 A	c	d	$c + d$
合计	$a + c$	$b + d$	N

a A 与 B 同时出现

b A 出现而 B 不出现

c B 出现而 A 不出现

d A 与 B 都不出现

$$N = a + b + c + d$$

若只有 d 变化, A 与 B 之间被测出的关系是否应当变化?

零不变性：改变 d，保持 a、b、c 固定



关系证据保持不变，但零事务数量在变。

数据集	a	b	c	d	N
D ₁	20	10	10	60	100
D ₂	20	10	10	960	1,000

度量	公式	D ₁	D ₂	是否零不变?
Jaccard	$a / (a + b + c)$	0.50	0.50	是
Kulczynski	$\frac{1}{2}[a/(a+b) + a/(a+c)]$	0.67	0.67	是
提升度	$aN / [(a+b)(a+c)]$	2.22	22.22	否

零不变的含义：仅改变 d 不会改变度量值。

哪些度量是零不变的？

□ 公式不依赖于 d 或 N 时，即为零不变。

度量	公式	是否零不变？	主要解释
Jaccard	$a / (a+b+c)$	是	含 A 或 B 的事务之间的重叠
余弦	$a / \sqrt{[(a+b)(a+c)]}$	是	归一化共现
Kulczynski	$\frac{1}{2}[a/(a+b)+a/(a+c)]$	是	两个方向置信度的平均
提升度	$aN / [(a+b)(a+c)]$	否	相对全体总体独立性的偏离
χ^2	观测计数 vs. 期望计数	否	依赖性的统计检验

零不变并不意味着普遍更好。每个度量回答不同的问题。

仅有零不变性还不够：Kulczynski 与 IR



Kulczynski 度量对称关联；IR（不平衡比）揭示支持度不平衡。

$$\text{Kulc}(A,B) = \frac{1}{2}[p + q]$$

$$p = P(B|A), \quad q = P(A|B)$$

$$\text{IR}(A,B) = |s(A) - s(B)| / [s(A) + s(B) - s(A \cap B)]$$

IR = 0: 支持度均衡
IR 接近 1: 严重不平衡

示例	a	b	c	p	q	Kulc	IR
均衡对	20	10	10	0.67	0.67	0.67	0.00
不平衡对	10	0	20	1.00	0.33	0.67	0.67

两个例子的 Kulc 都是 0.67，但第二对方向性很强且不平衡。

注：当项的支持度差异很大时，应把 IR 与 Kulczynski 一起报告。

为模式评估选择度量

□ 选择与分析问题及零情形含义相匹配的度量。

问题	建议度量	原因
活跃情形重叠多少？	Jaccard	忽略“两者皆无”的情形
归一化共现有多强？	余弦	对不同边际计数做调整
两个方向上的关联有多强？	Kulczynski + IR	把关联强度与不平衡结合起来
全体总体距独立性有多远？	提升度	使用边际概率与 N
依赖性在统计上是否显著？	χ^2	使用期望计数与样本量

稀疏数据；零总体不稳定

优先使用零不变度量

稳定、有意义的零总体

提升度或卡方可能合适

快速检查

问题：规则 $X \rightarrow Y$ 的置信度为 90%，但提升度 ≈ 1 。最好如何解释？

A	B	C	D
X 强烈导致 Y	Y 很常见, X 几乎不增加信息	该规则一定很罕见	Apriori 把支持度计错了

答案：B。

$\text{lift} = \text{confidence} / s(Y) \approx 1$ 。因此， $\text{confidence} = P(Y|X) \approx s(Y) = P(Y)$ 。观察到 X 并不会实质改变 Y 的概率。置信度必须相对后件的基率来解释。

第2部分小结

频繁模式挖掘需要分别决定计算、输出与解释。

1 高效搜索

- Apriori 与 DHP 逐层生成候选并尽早剪枝。
- ECLAT 用 TID 集合交运算做深度优先搜索。
- FP-growth 挖掘条件 FP-tree，而不显式生成候选。

2 紧凑输出

- 闭项集去掉相同支持度的冗余，同时保留支持度信息。
- 极大项集压缩更强，但丢弃精确的子集支持度。

3 有意义的评估

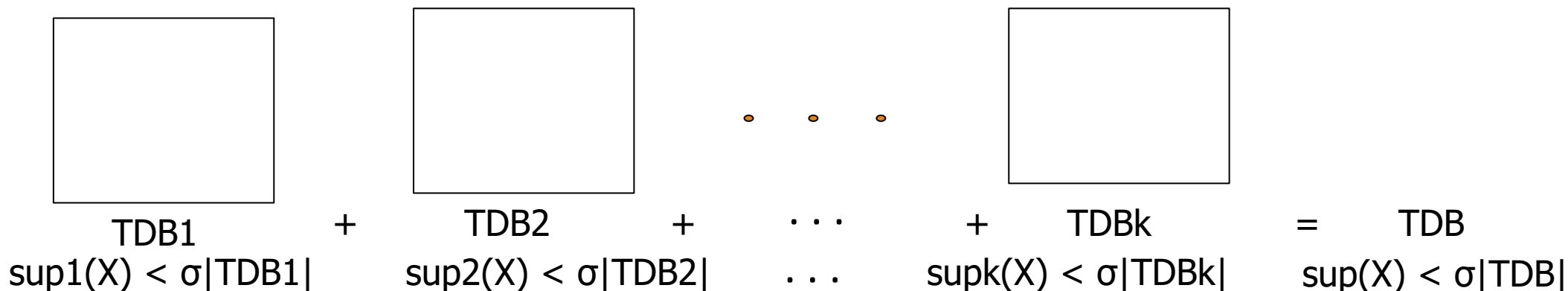
- 支持度与置信度描述频率，但并不确立依赖性。
- 提升度把置信度与后件的基率相比较。
- 当零事务不稳定时，使用零不变度量。

附录

第2部分备用幻灯片

划分法：数据库只扫描两遍

- **定理：在 TDB 中潜在频繁的任何项集，必在 TDB 的至少一个划分中频繁**



- **方法：扫描数据库两遍 (A. Savasere, E. Omiecinski and S. Navathe, VLDB'95)**

- **扫描 1：划分数据库，使每个划分能放入主存**

- 在该划分中挖掘局部频繁模式

- **扫描 2：汇总全局频繁模式**

- 找出全局频繁项集候选（至少在一个划分中频繁者）

- 再扫描一次 TDB_i，求这些候选的真实频率

(为什么?)

直接散列与剪枝 (DHP)

- 散列: $v = \text{hash}(\text{itemset})$
- 第 1 遍扫描: 计数 1-项集时, 对 2-项集散列以计算桶计数
- 示例: 在 TDB 的第 1 遍扫描中, 计数 1-项集, 并把事务中的 2-项集散列到其桶
 - 桶 v_1 : {ab, ad, ce}
 - 桶 v_2 : {bd, be, de}
 - 桶 v_n : ...
- 第一遍扫描结束时,
 - 若 $\text{minsup} = 80$, 剪掉 ab, ad, ce, 因为它们共享的桶 {ab, ad, ce} 计数 < 80

不同项集的 v 可能相同

项集	计数
{ab, ad, ce}	35
{bd, be, de}	298
.....	...
{yz, qs, wt}	58

散列表



检查 minsup

关键观察: 若对应散列桶计数低于 minsup 阈值, 则 k -项集不可能频繁

桶计数低即可保证不频繁; 桶计数高只说明有可能。

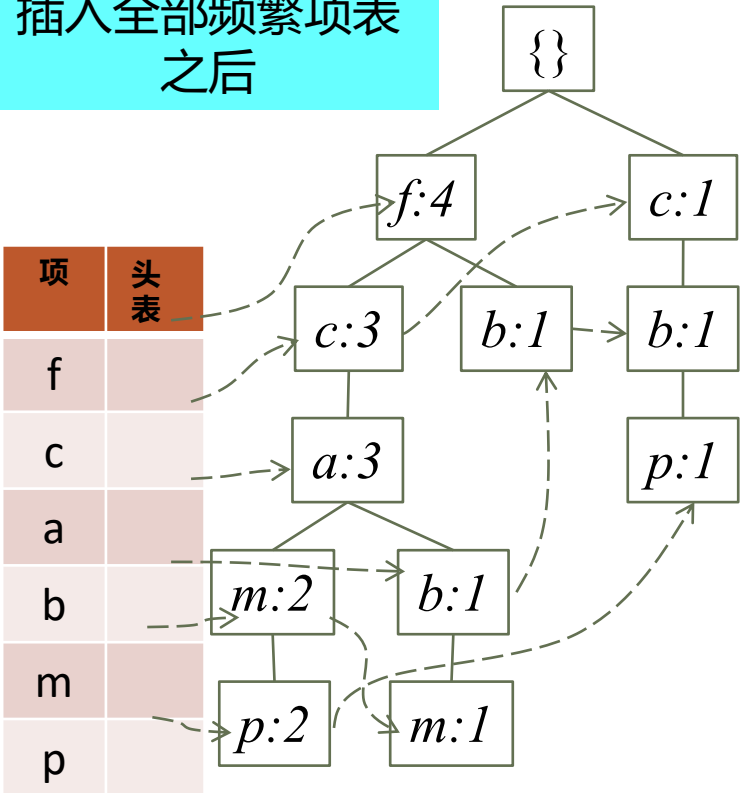
更多 Apriori 变体

方法	核心思想
动态项集计数	在一次数据库扫描中的检查点引入候选
抽样	用放宽阈值挖掘样本，再在全库上验证候选
支持度下界	用安全界在精确计数前淘汰候选
树投影	把事务投影到针对搜索的树中
H-Miner	用超链接数组避免反复构造投影数据库
LCM / 超立方体分解	用专门的闭包与前缀技术枚举频繁或闭项集

这些变体以不同方式减少扫描、候选或表示代价。

通过前缀合并构造 FP-Tree

插入全部频繁项表之后



插入的事务	对树的影响
f, c, a, m, p	创建路径 f:1, c:1, a:1, m:1, p:1
f, c, a, b, m	把 f-c-a 增至 2, 再分支到 b-m
f, b	增加 f, 再添加第二个 b 结点
c, b, p	创建以 c 开头的新根分支
f, c, a, m, p	增加完整的第一条路径

树合并前缀；它不会合并出现在不同前缀下的相同项。

公式速查

量	公式
支持度计数	$\sigma(X) = \text{包含 } X \text{ 的事务数}$
相对支持度	$s(X) = \sigma(X) / N$
置信度	$\text{conf}(X \Rightarrow Y) = s(X \cup Y) / s(X)$
提升度	$\text{lift}(X \Rightarrow Y) = \text{conf}(X \Rightarrow Y) / s(Y)$
期望计数	$E_{ij} = \text{行合计} \times \text{列合计} / \text{总计}$
χ^2	$\chi^2 = \sum (O - E)^2 / E, \text{ df} = (r - 1)(c - 1)$
Kulczynski	$\text{Kulc}(A, B) = 1/2 [P(B A) + P(A B)]$

解释概率之前先说清分母。

提升度：演算示例

列联表展示两个分类变量在所有组合上如何共现。

会员类型	短时	非短时	行合计
订阅用户	50	10	60
临时用户	30	10	40
列合计	80	20	100

行描述会员类型；列描述时长类别。

提升度：演算示例

$\text{lift}(\text{订阅用户} \Rightarrow \text{短时}) \approx 1.04$

提升度值	解释
> 1	相对独立的正关联
$= 1$	独立基线
< 1	相对独立的负关联

提升度度量相对独立性的乘性偏离，而不是因果效应。

卡方、自由度与 p 值

$$\chi^2 = \sum (O - E)^2 / E$$

$$df = (r - 1)(c - 1) = 1$$

单元格	O	E	(O-E) ² /E
订阅用户, 短时	50	48	0.083
订阅用户, 非短时	10	12	0.333
临时用户, 短时	30	32	0.125
临时用户, 非短时	10	8	0.500
合计			$\chi^2 \approx 1.04$

$\chi^2(df=1)$ 分布右尾面积
超过 1.04

$p \approx 0.31$

没有反对独立性的强证据

p 值不是原假设为真的概率。

趣味性度量与零不变性

- 零不变性的含义：零事务的数量无关紧要（不改变度量值）。
- 若干趣味性度量：有些是零不变的

Measure	Definition
$\chi^2(A, B)$	$\sum_{i,j} \frac{(e(a_i, b_j) - o(a_i, b_j))^2}{e(a_i, b_j)}$
$Lift(A, B)$	$\frac{s(A \cup B)}{s(A) \times s(B)}$
$Allconf(A, B)$	$\frac{s(A \cup B)}{\max\{s(A), s(B)\}}$
$T_{\text{max}}(A, B)$	$s(A \cup B)$

Let

$$p = \frac{s(A \cup B)}{s(A)} = P(B|A)$$

$$q = \frac{s(A \cup B)}{s(B)} = P(A|B)$$

p, q are null invariant

Essentially min, max,
mean variants of p, q

零不变性：一个重要性质

□ 为何零不变性对海量事务数据分析至关重要？

□ 许多事务可能既不含牛奶也不含咖啡！

牛奶 vs. 咖啡列联表

	<i>milk</i>	$\neg milk$	Σ_{row}
<i>coffee</i>	<i>mc</i>	$\neg mc$	<i>c</i>
$\neg coffee$	<i>m</i> $\neg c$	$\neg m$ $\neg c$	$\neg c$
Σ_{col}	<i>m</i>	$\neg m$	Σ

- 提升度与 χ^2 不是零不变的：加入大量“两者皆无”事务时，其取值可能改变。
- 许多关联度量都不是零不变的。

零事务
相对于 *m* 与 *c*

Data set	<i>mc</i>	$\neg mc$	<i>m</i> $\neg c$	$\neg m$ $\neg c$	χ^2	<i>Lift</i>
<i>D</i> ₁	10,000	1,000	1,000	100,000	90557	9.26
<i>D</i> ₂	10,000	1,000	1,000	100	0	1
<i>D</i> ₃	100	1,000	1,000	100,000	670	8.44
<i>D</i> ₄	1,000	1,000	1,000	100,000	24740	25.75
<i>D</i> ₅	1,000	100	10,000	100,000	8173	9.18
<i>D</i> ₆	1,000	10	100,000	100,000	965	1.97

零不变度量的比较

- 并非所有零不变度量都同等
- 哪一个更好?
 - D4—D6 能区分这些零不变度量
 - Kulc (Kulczynski 1927) 表现稳健, 并对两个方向蕴涵保持平衡

两变量列联表

	<i>milk</i>	$\neg milk$	Σ_{row}
<i>coffee</i>	<i>mc</i>	$\neg mc$	<i>c</i>
$\neg coffee$	$m \neg c$	$\neg m \neg c$	$\neg c$
Σ_{col}	<i>m</i>	$\neg m$	Σ

全部 5 个都是零不变的

Data set	<i>mc</i>	$\neg mc$	$m \neg c$	$\neg m \neg c$	<i>AllConf</i>	Jaccard	<i>Cosine</i>	<i>Kulc</i>	<i>MaxConf</i>
<i>D</i> ₁	10,000	1,000	1,000	100,000	0.91	0.83	0.91	0.91	0.91
<i>D</i> ₂	10,000	1,000	1,000	100	0.91	0.83	0.91	0.91	0.91
<i>D</i> ₃	100	1,000	1,000	100,000	0.09	0.05	0.09	0.09	0.09
<i>D</i> ₄	1,000	1,000	1,000	100,000	0.5	0.33	0.5	0.5	0.5
<i>D</i> ₅	1,000	100	10,000	100,000	0.09	0.09	0.29	0.5	0.91
<i>D</i> ₆	1,000	10	100,000	100,000	0.01	0.01	0.10	0.5	0.99

微妙之处: 在这些情形上它们意见不一

零不变度量族

度量	公式	解释
Jaccard	$a / (a+b+c)$	全部非零事务中的共享正例
余弦	$a / \sqrt{(a+b)(a+c)}$	两个支持度的几何平衡
全置信度	$\min[a/(a+b), a/(a+c)]$	较弱的方向置信度
最大置信度	$\max[a/(a+b), a/(a+c)]$	较强的方向置信度
Kulczynski	$\frac{1}{2}[a/(a+b) + a/(a+c)]$	两个方向的平均
IR	$ b-c / (a+b+c)$	边际支持度不平衡

零不变性意味着“两者皆无”计数 d 不改变该度量。

Kulczynski 度量下的不平衡比

IR（不平衡比）：度量规则蕴涵中两项集 A 与 B 的不平衡：

$$IR(A, B) = \frac{|s(A) - s(B)|}{s(A) + s(B) - s(A \cup B)}$$

Kulczynski 与不平衡比（IR）放在一起，能把 D4 到 D6 三个数据集说清楚

- D4 中性且均衡；D5 中性但不平衡
- D6 中性但非常不平衡

Data set	mc	$\neg mc$	$m \neg c$	$\neg m \neg c$	Jaccard	Cosine	Kulc	IR
D_1	10,000	1,000	1,000	100,000	0.83	0.91	0.91	0
D_2	10,000	1,000	1,000	100	0.83	0.91	0.91	0
D_3	100	1,000	1,000	100,000	0.05	0.09	0.09	0
D_4	1,000	1,000	1,000	100,000	0.33	0.5	0.5	0
D_5	1,000	100	10,000	100,000	0.09	0.29	0.5	0.89
D_6	1,000	10	100,000	100,000	0.01	0.10	0.5	0.99

示例：分析 DBLP 合著关系

- DBLP：计算机科学研究论文书目数据库

- 超过 380 万条关于作者、论文、会议/期刊、年份及其他信息的记录

ID	Author A	Author B	$s(A \cup B)$	$s(A)$	$s(B)$	Jaccard	Cosine	Kulc
1	Hans-Peter Kriegel	Martin Ester	28	146	54	0.163 (2)	0.315 (7)	0.355 (9)
2	Michael Carey	Miron Livny	26	104	58	0.191 (1)	0.335 (4)	0.349 (10)
3	Hans-Peter Kriegel	Joerg Sander	24	146	36	0.152 (3)	0.331 (5)	0.416 (8)
4	Christos Faloutsos	Spiros Papadimitriou	20	162	26	0.119 (7)	0.308 (10)	0.446 (7)
5	Hans-Peter Kriegel	Martin Pfeifle	18	146	18	0.123 (6)	0.351 (2)	0.562 (2)
6	Hector Garcia-Molina	Wilburt Labio	16	144	18	0.110 (9)	0.314 (8)	0.500 (4)
7	Divyakant Agrawal	Wang Hsiung	16	120	16	0.133 (5)	0.365 (1)	0.567 (1)
8	Elke Rundensteiner	Murali Mani	16	104	20	0.148 (4)	0.351 (3)	0.477 (6)
9	Divyakant Agrawal	Oliver Po	12	120	12	0.100 (10)	0.316 (6)	0.550 (3)
10	Gerhard Weikum	Martin Theobald	12	106	14	0.111 (8)	0.312 (9)	0.485 (5)

导师-学生关系：Kulc 高, Jaccard 低, cosine 居中

- 哪些作者对关系很强？A 是导师还是学生？

- 用 Kulc 发现导师-学生、密切合作者

为模式评估选择度量

- 稀疏数据中零事务很常见：多数购物篮可能两者都不含。
- 当“两者皆无”有意义且不占压倒性时，使用提升度与卡方。
- 当零计数主要反映抽样时，使用零不变度量。
- 把 Kulczynski 与 IR 一起用，以分离关联强度与方向不平衡。

应用：在 DBLP 中挖掘频繁合作者群体，并考察随时间可能的导师-学生方向。

推荐阅读（高效模式挖掘方法）

- ❑ R. Agrawal and R. Srikant, “Fast algorithms for mining association rules”, VLDB'94
- ❑ A. Savasere, E. Omiecinski, and S. Navathe, “An efficient algorithm for mining association rules in large databases”, VLDB'95
- ❑ J. S. Park, M. S. Chen, and P. S. Yu, “An effective hash-based algorithm for mining association rules”, SIGMOD'95
- ❑ S. Sarawagi, S. Thomas, and R. Agrawal, “Integrating association rule mining with relational database systems: Alternatives and implications”, SIGMOD'98
- ❑ M. J. Zaki, S. Parthasarathy, M. Ogihara, and W. Li, “Parallel algorithm for discovery of association rules”, Data Mining and Knowledge Discovery, 1997
- ❑ J. Han, J. Pei, and Y. Yin, “Mining frequent patterns without candidate generation”, SIGMOD'00
- ❑ M. J. Zaki and Hsiao, “CHARM: An Efficient Algorithm for Closed Itemset Mining”, SDM'02
- ❑ J. Wang, J. Han, and J. Pei, “CLOSET+: Searching for the Best Strategies for Mining Frequent Closed Itemsets”, KDD'03
- ❑ C. C. Aggarwal, M.A., Bhuiyan, M. A. Hasan, “Frequent Pattern Mining Algorithms: A Survey”, in Aggarwal and Han (eds.): Frequent Pattern Mining, Springer, 2014

推荐阅读（模式评估）

- ❑ C. C. Aggarwal and P. S. Yu. A New Framework for Itemset Generation. PODS'98
- ❑ S. Brin, R. Motwani, and C. Silverstein. Beyond market basket: Generalizing association rules to correlations. SIGMOD'97
- ❑ M. Klemettinen, H. Mannila, P. Ronkainen, H. Toivonen, and A. I. Verkamo. Finding interesting rules from large sets of discovered association rules. CIKM'94
- ❑ E. Omiecinski. Alternative Interest Measures for Mining Associations. TKDE'03
- ❑ P.-N. Tan, V. Kumar, and J. Srivastava. Selecting the Right Interestingness Measure for Association Patterns. KDD'02
- ❑ T. Wu, Y. Chen and J. Han, Re-Examination of Interestingness Measures in Pattern Mining: A Unified Framework, Data Mining and Knowledge Discovery, 21(3):371-397, 2010